Microcontroller (MCU) and Pulse-Width Modulation (PWM)

By: >_ terminal gremlin (c-lydia ^-^)

Why Control System needs Electronics

Microcontrollers operate strictly as low-power digital logic devices. Their internal silicon consists of microscopic CMOS (Complementary Metal-Oxide Semiconductor) Transistors optimized for digital computation, state evaluation, and high-speed switching between logic LOW ($0V$) and HIGH ($3.3V$). They cannot directly power actuators.

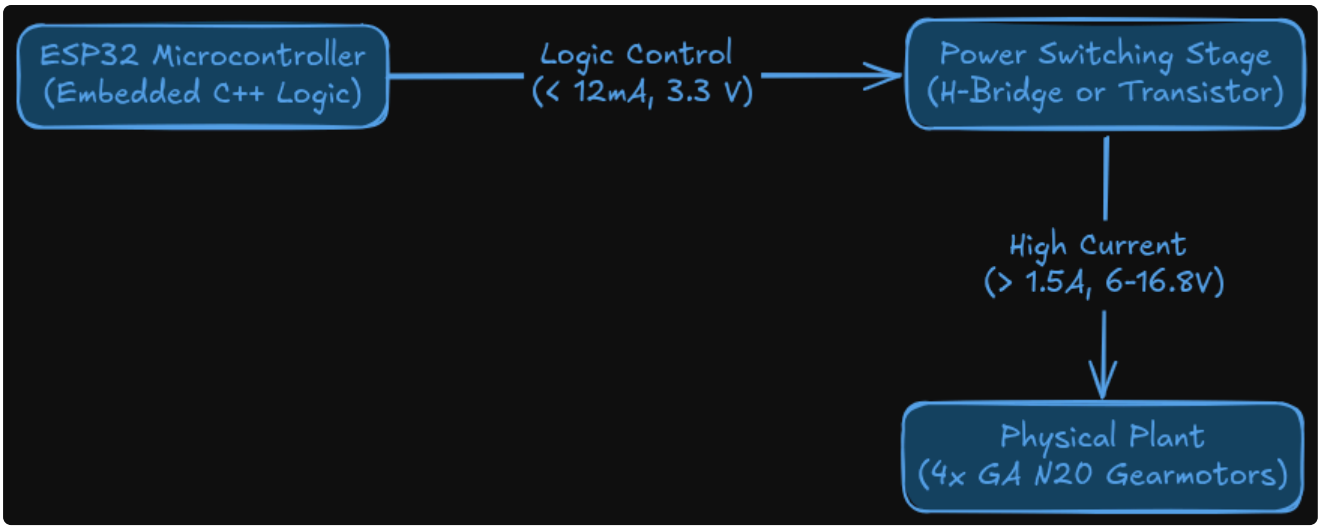
The Current and Voltage Gap

Connecting an electromechanical load directly to a microcontroller pin violates electrical boundaries:

- **Microcontroller Output Drive Limits:** an ESP32 GPIO (General-Purpose Input/Output) pin can safely source or sink a nominal current of $12mA$ (recommended operating target), with an absolute maximum silicon limit of $40mA$ per pin and a cumulative chip-wide ceiling of $120mA$.
- **Actuator Current Demands:** the 4x GA N20 $6V$ $300RPM$ micro metal gearmotors draw approximately $60mA$ to $120mA$ each under no-load conditions, $250mA$ to $400mA$ stall current per motor (approaching $5.0A$ total for a 4 wheel differential chassis stall).
- **Electrical Destruction:** direct connection of an electric motor to an MCU GPIO pin instantly destroys the internal output (MOSFETs: Metal-Oxide-Semiconductor Field-Effect Transistors). Destruction occurs through two simultaneous mechanisms:
  - *Thermal Overcurrent:* current demand exceeds the silicon trace capacity by more than an order of magnitude, causing thermal runaway and internal bonding-wire fusion.
  - *Inductive Kickback Voltage Spikes:* when current flowing through an inductive motor winding (L) is rapidly interrupted, the collapsing magnetic field induces a reverse electromotive force (back-EMF) governed by

    $$V_{spike} = -L\frac{dI}{dt}$$

    Because dt approaches zero during rapid solid state switching, this induced voltage spike frequently exceeds dozens or hundreds of volts, puncturing the thin gate oxide layer of the microcontroller's internal transistors.



Logic Level Domains and Signal Translation

Mobile robot platforms operate across multiple incompatible voltage domains. You must prevent cross-domain overvoltage faults:

$3.3V$ CMOS Logic Domain (ESP32)

- $V_{IL} \leq 0.8V$ (Maximum input voltage guaranteed to be read as logic LOW)
- $V_{IH} \geq 2.0V$ (Minimum input voltage guaranteed to be read as logic HIGH)
- $V_{OL} \leq 0.4V$ (Maximum output voltage when driving logic LOW)
- $V_{OH} \geq 2.64V$ (Minimum output voltage when driving logic HIGH)

$5.0V$ TTL (Transistor-Transistor Logic) or CMOS Domain (HC-SR04 Ultrasonic Sensor, Legacy Peripherals)

- An ESP32 $3.3V$ output can typically drive a $5V$ TTL input directly because $V_{OH} \geq 2.64V$ comfortably exceeds the $5V$ TTL V_{IH} threshold of $2.0V$
- Conversely, a $5V$ sensor output connected directly to an ESP32 GPIO pin will exceed the silicon maximum tolerance ($V_{DD} + 0.5V \approx 3.6V$), permanently damaging the pin. Passive resistive dividers or active bidirectional logic-level shifters must be inserted

$6.0V$ Motor Drive Rail (Buck Converter 2)

- Delivers regulated high current to the motor armatures via the red dual H-bridge motor drivers. Completely isolated from the logic power plane.

$14.8V$ Nominal or $16.8V$ Peak Domain (4S 18650 Battery Pack)

- Primary energy storage. Stepped down via switching buck regulators. Must never connect directly to logic or motor pins without regulation

Grounding, Return Paths, and Decoupling

Common ground is an absolute electrical requirement: when two distinct subsystems exchange logical signals, they must share an unbroken, low-impedance ground reference. Without a common ground reference, a signal defined as $3.3V$ by the transmitter has an undefined, floating voltage potential at the receiver.

However, sharing a ground reference does not permit routing heavy motor return currents through breadboard logic tracks:

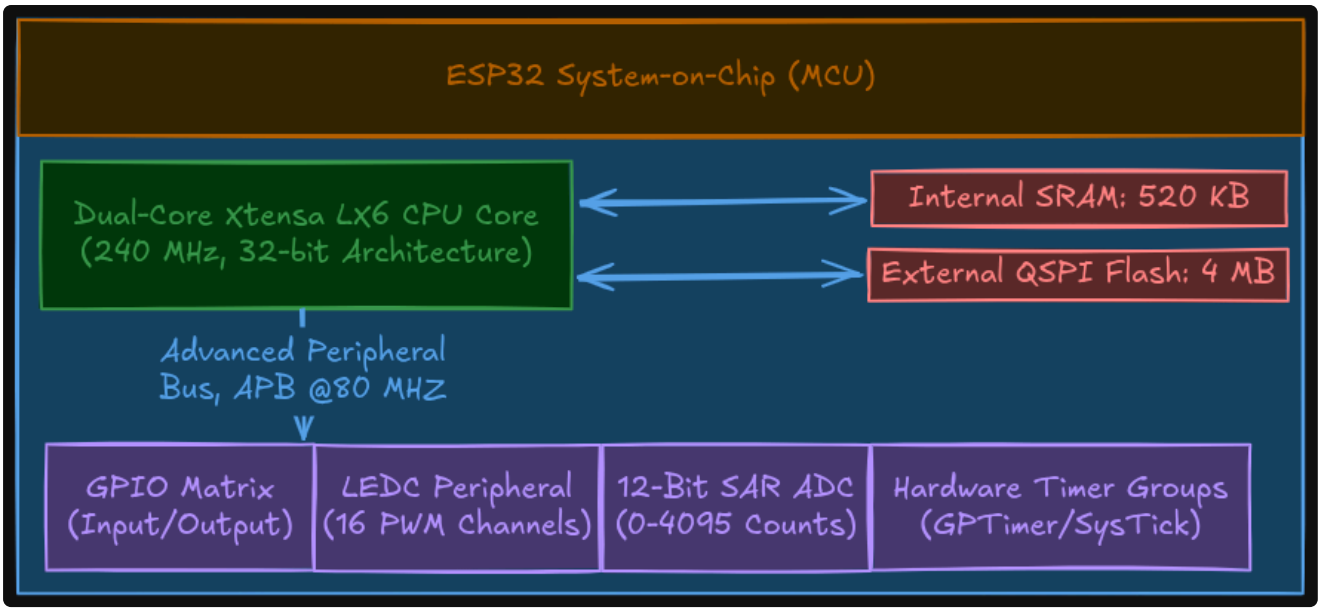
- **Ground loop interference:** motor armatures switching multiple amperes introduce high-frequency ground bounce and inductive noise across ground leads.
- **Separation Rule:** motor current paths must be kept physically short, wired with heavy gauge wire (AWG 18/20), and terminated directly at the power supply or screw terminal star ground.
- **Decoupling capacitors:** high-speed digital logic requires local charge reservoirs. A $100n$ ceramic capacitor placed immediately adjacent to the MCU power pins shunts high-frequency noise to ground, while a parallel 100μ electrolytic capacitor buffers transient supply dips.

Microcontroller Architecture and Peripheral Hardware

Understanding the physical silicon inside the microcontroller prevents software-induced hardware failures.

Architectural classification: CPU, MPU, and MCU

- **Central Processing Unit (CPU) or Core:** the computational arithmetic-logic unit (ALU) and instruction execution pipeline that fetches, decodes, and executes machine instructions.
- **Microprocessor (MPU):** a high-performance processor (e.g., ARM Cortex-A53, Intel Core i7) lacking on-chip RAM, Flash, or power regulation. It requires external DDR memory and runs general-purpose multi-tasking operating systems (Linux, ROS 2). It excels at heavy cognitive computation (computer vision, SLAM, point-cloud processing) but lacks deterministic, hard real-time pin timing.
- **Microcontroller (MCU):** a self-contained System-on-Chip (SoC) integrating a CPU core with high-speed generation, and dedicated hardware peripherals (GPIO, hardware timers, ADC, UART, I2C, SPI, CAN). It provides deterministic microsecond-level hardware control.



ESP32 System-on-Chip Specification

The ESP32-WROOM-32 DevKit is built around the Espressif ESP32-D0WDQ6 chip:

- **Processing Cores:** dual-core Tensilica Xtensa 32-bit LX6 processors operating at $240MHz$ (delivering up to $600DMIP$).
- **Bus Architecture:** internal Advanced High-performance Bus (AHB) for memory operations and Advanced Peripheral Bus (APB) operating at a fixed $80MHz$ serving as the reference timebase for hardware timers, communication engines, and PWM modules
- **Internal Memory:** $520KB$ internal on-chip SRAM allocated for runtime stack, heap, and DMA buffers, paired with $4MB$ external QSPI Flash memory mapped into CPU instruction space

GPIO Matrix and Pin Constraints

The ESP32 features a flexible internal crossbar multiplexer known as GPIO Matrix. However, physical pins have rigid electrical limitations:

- **Standard Bidirectional GPIOs (`GPIO 0-33`):** fully configurable with software-programmable push-pull output drivers and weak internal pull-up or pull-down resistors (approximately $45k\Omega$)
- **Input-Only GPIO Pins (`GPIO 34, 35, 36, 39`):** these pins **lack internal output driver transistors, pull-up resistors, and pull-down resistors**. They can only be utilized as high-impedance digital or analog inputs (such as battery voltage monitoring or analog sensor lines)
- **Boot Strapping Pins (`GPIO 0, 2, 5, 12, 15`):** sampled during initial power-on reset (POR) to determine boot mode (UART download and Flash boot). Pulling GPIO 12 HIGH

externally at boot will cause Flash voltage regulator failure ($1.8V$ and $3.3V$ mismatch), causing the MCU to hang

Floating Inputs and Pull-up or Pull-down Selection

An input pin configured without pull resistors (`INPUT`) presents an input impedance exceeding $10M\Omega$. In this high-impedance state, stray electromagnetic radiation (ambient EMI from fluorescent lights, nearby motor commutators, or AC power cords) couples capacitively into the pin gate, causing the measured logic level to oscillate unpredictably.

- **`pinMode(pin, INPUT_PULLUP)`:** connects an internal $\approx 45k\Omega$ resistor to $3.3V$. The pin defaults to logic HIGH ($3.3V$); closing a switch connected to Ground pulls the pin to logic LOW ($0V$, Active-LOW logic)
- **`pinMode(pin, INPUT_PULLDOWN)`:** connects an internal $\approx 45k\Omega$ resistor to Ground. The pin defaults to logic LOW ($0V$); closing a switch connected to $3.3V$ pulls the pin to logic HIGH ($3.3V$, Active-HIGH logic)

Pulse-Width Modulation and LEDC

Pulse-Width Modulation is the standard method for controlling analog power delivery to motors and actuators using binary digital outputs. By rapidly toggling a pin between $0V$ and $3.3V$ at a constant carrier frequency, an electromechanical load acts as a low-pass filter, integrating the discrete voltage pulses into an effective **time-averaged DC voltage (V_{avg})**.

PWM waveform

A PWM waveform is defined by:

- **Carrier Period (T):** the duration of one complete waveform cycle

$$T = T_{ON} + T_{O_{\text{ff}}} = \frac{1}{f_{PM}}$$

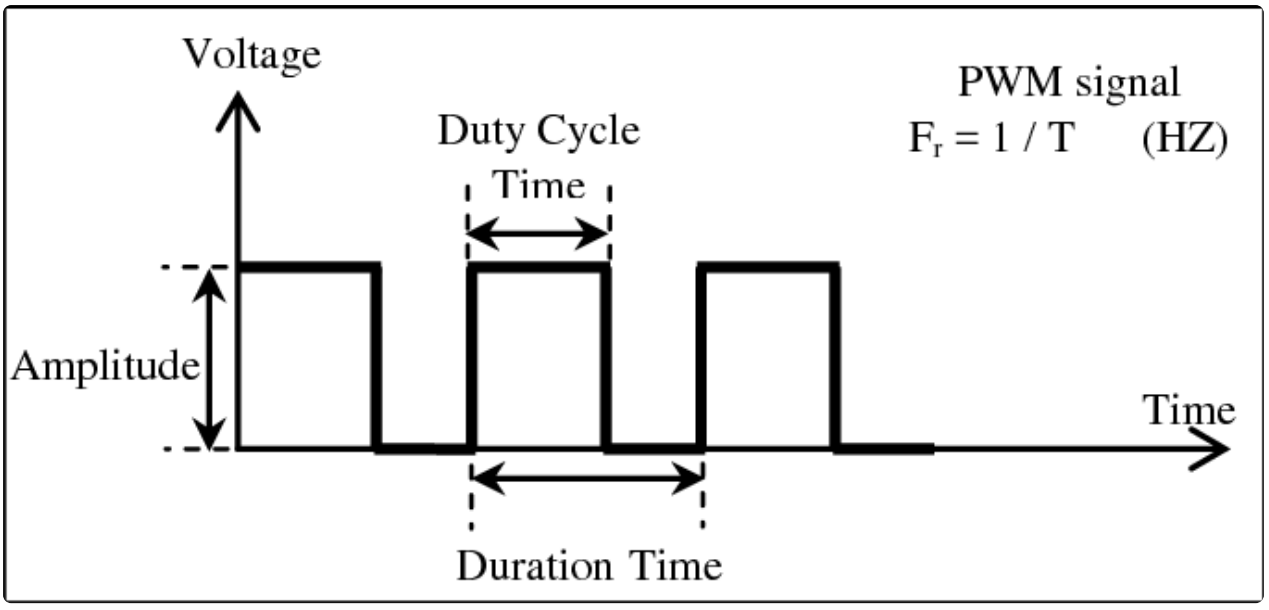
- **Carrier Frequency (f_{PM}):** the pulse repetition rate per second (Hertz)
  - Low carrier frequencies ($< 1kHz$) cause severe torque pulsation and high mechanical vibration in small motor gearboxes
  - Audible frequencies ($1kHz - 16kHz$) excite the mechanical natural frequencies of motor laminations, creating an irritating acoustic whine
  - Ultrasonic frequency ($20kHz$) pushes switching beyond human hearing limits ($20Hz - 20kHz$), delivering completely silent motor actuation
- **Duty Cycle (D):** the fractional ratio of active HIGH duration (T_{ON}) relative to total period (T)

$$D = \frac{T_{ON}}{T}$$

$$D_{percentage} = D \cdot 100$$

- **Time-Averaged DC Voltage (V_{avg}):** the mathematical definite integral over one complete cycle

$$V_{avg} = \frac{1}{T_0} \int_0^T V(t)dt = \frac{1}{T}(V_{HIGH} \cdot T_{ON} + 0 \cdot T_{OFF}) = D \cdot V_{HIGH}$$



ESP32 LEDC (LED Control) Hardware Peripheral

Unlike legacy software PWM implementations (which rely on CPU timer interrupts that introduce severe timing jitter whenever serial communication or Wi-Fi interrupts fire), the ESP32 incorporates a dedicated **LEDC hardware peripheral**:

- **Hardware Architecture:** 16 independent PWM channels grouped into 2 high-speed or low-speed blocks, driven by 4 internal hardware timer modules
- **Clock Source:** driven directly by the $80MHz$ APB clock (`LEDC_AUTO_CLK` or `LEDC_APB_CLK`). Once configured, the peripheral operates autonomously via direct memory-mapped registers, consuming zero CPU cycles
- **Resolution (N bits):** sets the integer quantization range of the duty cycle

$$steps = 2^N$$

  - For $N = 8bits$, duty cycle integers range from 0 to 255 (256 discrete levels)
  - For $N = 10bits$, duty cycle integers range from 0 to 1023 (1024 discrete levels)
- **Frequency and Resolution Trade-off:** the maximum achievable PWM frequency is constrained by the timer clock and bit resolution

$$f_{PWM,max} = \frac{f_{APB}}{2^N} = \frac{80MHz}{2^N}$$

At $8bits$ (256 counts),

$$f_{PM,max} = \frac{80,000,000Hz}{256} = 312.5kHz$$

Selecting $f_{PM} = 20kHz$ at $8bits$ resolution operates well within hardware limits, ensuring stable counter reloading and jitter-free output

How to test hardware

How to use Multimeter

The Digital Multimeter consists of 3 modes

Mode 1: Continuity Mode

- Keep the power disconnected
- Verify 0Ω continuity along common Ground (GND) across all breadboard tie-points
- Confirm zero short-circuit connection between VCC ($3.3V$ or $5V$) power rails and Ground

Mode 2: Resistance Mode

- Keep the power disconnected
- Verify passive component values (e.g., confirming 330Ω current-limiting resistor)
- Verify DC motor armature coil winding resistance ($> 1.0\Omega$, not a dead short)

Mode 3: DC Voltage Mode

- Connect to the power
- Measure $3.3V$ LDO regulator rail (tolerable window: $3.25V$ to $3.35V$)
- Measure $5.0V$ USB or buck converter rail (tolerable windows: $4.75V$ to $5.25V$)
- Probe PWM output pin to verify time-averaged DC voltage scaling

Instrument Impedance and Loading Effects

An ideal voltmeter possesses infinite input impedance ($R_{meter} = $), drawing zero current from the test circuit. However, in real tests:

- **Digital Multimeters:** incorporate active FET input amplifiers presenting an impedance of $10M\Omega$ at high-impedance nodes (such as MCU push-pull GPIOs or power rails); the meter draws negligible current ($I_{meter} = 3.3V/10M\Omega = 0.33\mu A$), causing insignificant circuit loading
- **Internal Analog Filter Mechanics:** when a multimeter is set to DC Voltage mode and connected to a $20kHz$ PWM signal, its internal dual-slope analog-to-digital converter acts as a continuous low-pass filter with an integration window typically between $100ms$

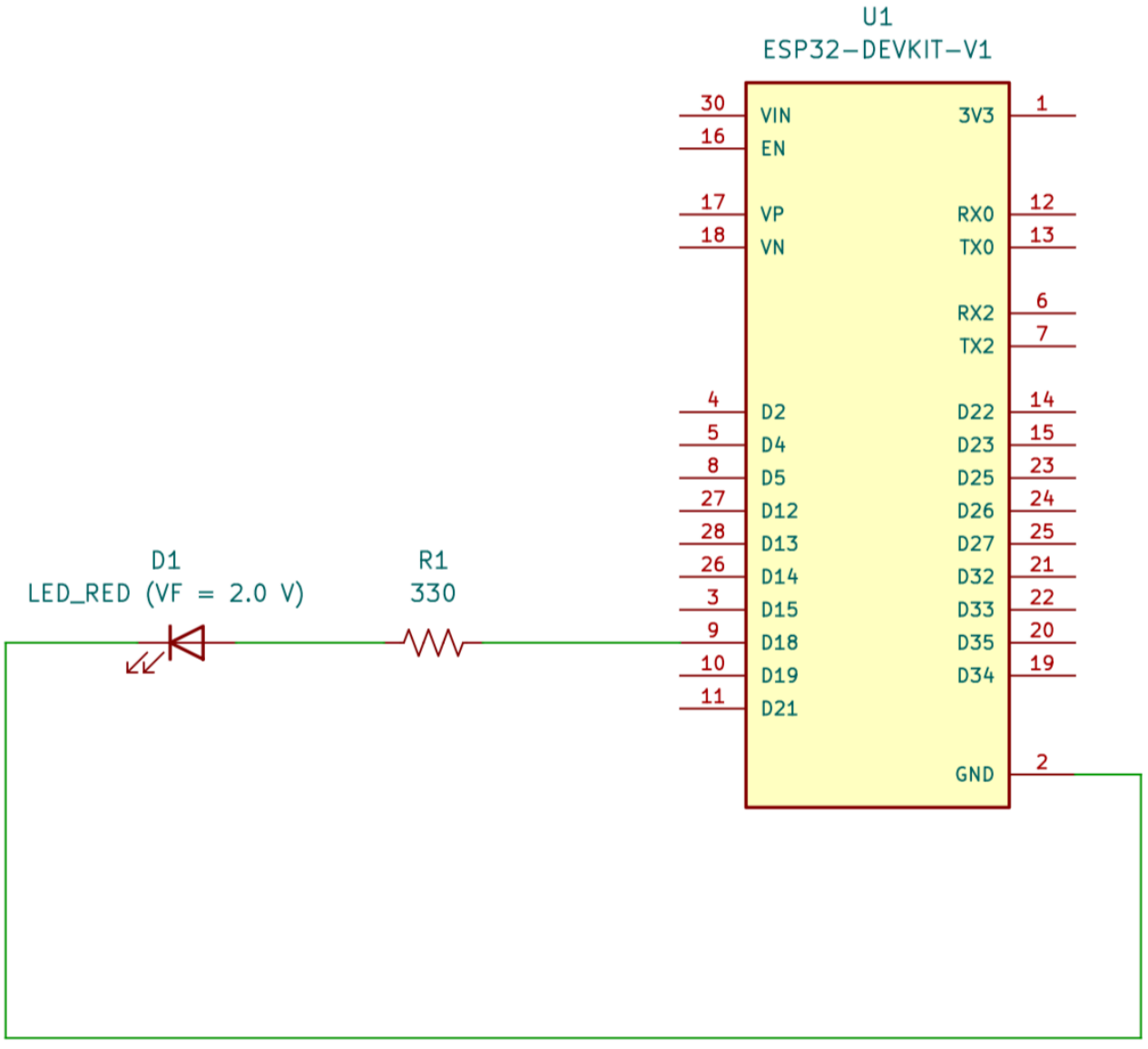
and $300ms$. Because the meter's filter bandwidth is orders of magnitude lower than the $20kHz$ carrier frequency, it rejects the alternating AC pulse train and measures the **exact mathematical DC average voltage**

$$V_{meter} = V_{avg} = \frac{\text{Duty Count}}{255} \cdot 3.3V$$

Implementation: PWM Generation

How to wire components

1. Insert the ESP32 DevKit into the breadboard, straddling the center divider so pins on opposite sides do not short together
2. Locate `GPIO 18` and `GND` on the ESP32 pinout diagram
3. Connect the 330Ω current-limiting resistor from `GPIO 18` to an unused breadboard row
4. Insert the LED: connect the longer lead (Anode, $+$) to the resistor row, and connect the short lead with the flat edge (Cathode, $-$) to the `GND` rail
5. Connect a black jumper wire from the breadboard `GND` rail to the ESP32 `GND` pin



Pre-Power Test (for Safety)

Do NOT plug in the USB cable until you complete these tests

1. Switch the Multimeter to **Continuity Mode** (beep symbol). Touch the two meter probes together to confirm audible beeper operation
2. Place the black probe on the ESP32 metal USB shield (chassis Ground). Place the red probe on the breadboard `GND` rail. The meter must beep, confirming a solid Ground reference
3. Switch the Multimeter to **Resistance Mode ($200k\Omega$ range)**. Measure between the ESP32 `3V3` pin and `GND`
   - A healthy unpowered board reads high impedance ($> 100k\Omega$)
   - A reading below 10Ω indicates an immediate solder bridge or power short circuit. **Do NOT apply USB power if a short is detected**

Firmware

Open Arduino IDE, select **Tools > Board > esp32 > ESP32 Dev Module**, and paste the following verified firmware sketch:

```
#include <Arduino.h>

// Hardware pin and LEDC configuration constants
const uint8_t PWM_OUTPUT_PIN = 18; // Target GPIO output pin
const uint32_t PWM_FREQUENCY = 20000; // 20 kHz ultrasonic carrier
(silent)
const uint8_t PWM_RESOLUTION = 8; // 8-bit duty resolution (0 - 255
counts)
const uint8_t LEDC_CHANNEL = 0; // Hardware LEDC channel index 0

// Test duty cycle array: 0%, 25%, 50%, 75%, 100%
const uint8_t TEST_DUTY_COUNTS[] = {0, 64, 128, 192, 255};
const uint8_t NUM_TEST_STEPS =
sizeof(TEST_DUTY_COUNTS)/sizeof(TEST_DUTY_COUNTS[0]);

void setup() {
    // Initialize USB UART console for debugging
    Serial.begin(115200);
    delay(500); // Allow UART hardware buffer to stabilize

    Serial.println("PWM TEST");

    // Configure and attach GPIO 18 to LEDC hardware timer
    bool attach_success = ledcAttachChannel(PWM_OUTPUT_PIN,
```

```
PWM_FREQUENCY, PWM_RESOLUTION, LEDC_CHANNEL);

    if (attach_success) {
        Serial.println("[STATUS] LEDC Initialized Successfully");
        Serial.printf("Target pin: GPIO %d | Freq: %d Hz | Resolution:
%d-bit\n", PWM_OUTPUT_PIN, PWM_FREQUENCY, PWM_RESOLUTION);
    } else {
        Serial.println("[FATAL ERROR] Failed to configure LEDC
channel");

        while (true) {
            delay(1000); // Halt execution if hardware timer allocation
fails
        }

        Serial.println("Starting 5-step duty cycle test (Hold: 8 seconds
per step)...\n");
    }
}

void loop() {
    for (uint8_t step = 0; step < NUM_TEST_STEPS; step++) {
        uint8_t current_duty = TEST_DUTY_COUNTS[step];

        // Write discrete duty cycle count directly to hardware timer
registers
        ledcWrite(PWM_OUTPUT_PIN, current_duty);

        // Compute electrical quantities
        float duty_percentage = ((float)current_duty/255.0f) * 100.0f;
        float dc_volts = ((float)current_duty/255.0f) * 3.300f;

        // Debugging
        Serial.printf("Step %d/%d: Duty Count = %3d/255 | Percentage =
%5.1f%%\n", step + 1, NUM_TEST_STEPS, current_duty, duty_percentage);
        Serial.printf("Calculated DC Voltage: %5.3f V\n", dc_volts);
        Serial.println(">>> PROBE GPIO 18 WITH MULTIMETER (DC VOLTS
MODE) NOW <<<");

        // Hold for 8 seconds to permit stable multimeter probe contact
        delay(8000);
    }
}
```

Verify data

1. Connect the USB cable to the PC. Open the Arduino Serial Monitor at `115200` baud.
2. Switch the Multimeter to **DC Voltage Mode ($20V$ range)**
3. Firmly clip the Multimeter black probe to the breadboard `GND` rail
4. Place the Multimeter red probe on the breadboard tie-point connecting `GPIO 18` and the 330Ω resistor
5. Compare the reading with the Serial output

> Measured voltages must track the calculated value $\pm 0.05V$. Minor deviations stem from USB $5V$ bus tolerance and internal LDO regulator loading

Debugging

When hardware outputs do not match expectations, follow these instructions:

LED Remains Dark Across All Steps

- Measure voltage directly across the LED leads with the Multimeter. If the meter reads $\approx 1.8V$ to $2.1V$ (standard forward voltage drop V_f of a red GaAsP LED) but emits no light, the diode is reversed. Invert the LED leads.
- If the voltage across the LED is $0.00V$, probe the MCU pin directly. If the MCU pin is toggling, the 330Ω resistor or breadboard track is open.

Serial Monitor Output Garbled or Blank

- Verify baud rate setting in the bottom right corner of the Serial Monitor window matches `115200`
- Press the physical `EN` (RST) button on the ESP32 board to restart the processor

IDE Reports "A fatal error occurred: Failed to connect to ESP32: No serial data received"

- Hold down the physical `BOOT` button (GPIO 0) on the ESP32 board during the "Connecting...." compilation phase until writing begins, then release

Component Safety

- **18650 Li-ion Cells:** inspect battery plastic wraps for tears or punctures. Store cells inside plastic cases. Never place loose cells in pockets or toolboxes with loose metal screws where battery terminals can short circuit.
- **N20 Gearmotors:** do not manually backdrive the gearboxes at high speed by hand. The internal brass spur gear teeth have micro-scale pitch ($0.2 modules$) and can strip under external torque.

References

- Making Embedded Systems: Design Patterns for Great Software by Elecia White
- The Art of Electronics: Student Manual by Thomas C. Hayes and Paul Horowitz
- Feedback Systems: An Introduction for Scientists and Engineers by Karl Johan Åström and Richard M. Murray
- Modern Robotics: Mechanics, Planning, and Control by Kevin M. Lynch and Frank C. Park
- ESP32 Arduino Core Documentation: LED Control (LEDC) by Espressif Systems
- ESP32 Arduino Core Documentation: GPIO by Espressif Systems
- Arduino Core for the ESP32 by Espressif Systems
- Motor Drivers Architecture and Overview by Texas Instruments